# REGULAR EXPRESSION IN PYTHON

# INTRODUCTION

- A **regular expression** is a **pattern** that describes a set of strings.
- It helps you check whether a string contains a specific pattern or extract and replace parts of it.
- **Importing Regex Module : ** import re
- Common Uses of Regex

| Purpose | Example |
|---|---|
| Validate | Email, phone number, password |
| Search | Find words, digits, dates in a text |
| Extract | Extract domain from URL, numbers from string |
| Replace | Replace whitespace, remove symbols |

# IMPORTANT Meta - CHARACTERS

| Symbol | Meaning |
|---|---|
| . | Any character except newline |
| ^ | Start of string |
| $ | End of string |
| * | 0 or more repetitions |
| + | 1 or more repetitions |
| ? | 0 or 1 repetition |
| \d | Digit (0–9) |
| \w | Word character (A–Z, a–z, 0–9, _) |
| \s | Space/whitespace |
| [] | Character set |
| () | Group |

# Search( )

- The re.search() function is used to return the first occurrence of the pattern.
- This function searches the entire string and returns a match object if a match is found.
- Otherwise, it returns None in case the match is not found.

Example:

```
from re import *
string = "Python is a general-purpose language"
matched_string = search(r'general',string)
print(matched_string)
print(matched_string.start())
print(matched_string.end())
print(matched_string.span())
```

# Compile( )

- Converts a regular expression **string pattern** into a **compiled regex object** that can be reused for matching multiple times.

Without Compile()

```
re.search(pattern, text)
re.match(pattern, text)
re.findall(pattern, text)
```

With Compile( )

```
regex = re.compile(pattern)
regex.search(text)
regex.match(text)
regex.findall(text)
```

- This improves:
  - ✔ **Readability**
  - ✔ **Performance** (faster if used repeatedly)

# Split( )

- used to **split a string based on a regex pattern**.
- split() splits a string **where the regular expression matches**, and returns a **list** of the resulting parts.
- Syntax : re.split(pattern, string, maxsplit=0)
- Example:

```
import re


text = "one-two-three-four"
result = re.split(r'-', text, maxsplit=2)
print(result)
```

# Sub( )

- sub() stands for **substitute**.
- Used to **search a pattern in a string and replace it with another string**.
- It replaces **all occurrences** of a regex pattern with a specified replacement.

**Syntax :** re.sub(pattern, replacement, string, count=0)

sub() is widely used for:

✔ cleaning text

✔ formatting strings

✔ removing unwanted characters

✔ replacing spaces, digits, symbols using regex

# Subn( )

- Used to **search a regex pattern, replace it with another string, and also return the number of replacements performed**.
- re.subn() works almost the same as re.sub(), but instead of returning only the modified string, it returns a **tuple**: (new_string, number_of_replacements)
- **Syntax :** re.subn(pattern, replacement, string, count=0)

# Escape ( )

- Used to **escape all special regex metacharacters in a string**, converting them into their **literal form**.
- re.escape() takes a string and **adds a backslash (\) before every regex metacharacter**, so the string can be safely used as a **literal pattern** in regex.

**Why do we need escape()?**

Regex contains special characters like: . ^ $ * + ? { } [ ] ( ) | \

If these appear in a string and we want them to be treated **as normal characters**, we need to escape them.

re.escape() does this **automatically**.

**Syntax : re.escape(string)**

# MATCH OBJECTS

- **Match object** represents the result of a successful pattern match.
- They return a **Match object** containing information about **where and what** was matched.
- A Match object stores detailed information about the match, including:
  - The actual substring that matched the pattern
  - The start and end index positions
  - The original searched string
  - Any captured groups inside parentheses ()
- You can use Match object methods to extract this information.

END..